

MINISTRY OF EDUCATION, CULTURE AND RESEARCH

OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Cuibari Vladislav FAF-243

Report

Laboratory Work No. 5

Operating Systems

Topic: The File System with Your Own Hands

Checked by:

Reițman Patricia, university assistant

Chișinău – 2026

INTRODUCTION

This laboratory examines the Linux file system from an ordinary user account in an Ubuntu 26.04.1 VirtualBox guest. I inspected file types, compared names with inodes, tested hard and symbolic links, changed permissions, and measured the guest disk and kernel-provided /proc files. The commands and results below come from my recorded terminal session.

1 THE TREE AND FILE TYPES

This chapter follows Part 1 of the handout. I inspected the directory tree, used the first character of ls -l to classify paths, and compared two system-directory sizes.

1.1 Tree and file types

```
whoami ls -ld /etc /etc/passwd /bin /dev/null /tmp
```

The evidence for this exercise is shown in Figure 1.1.

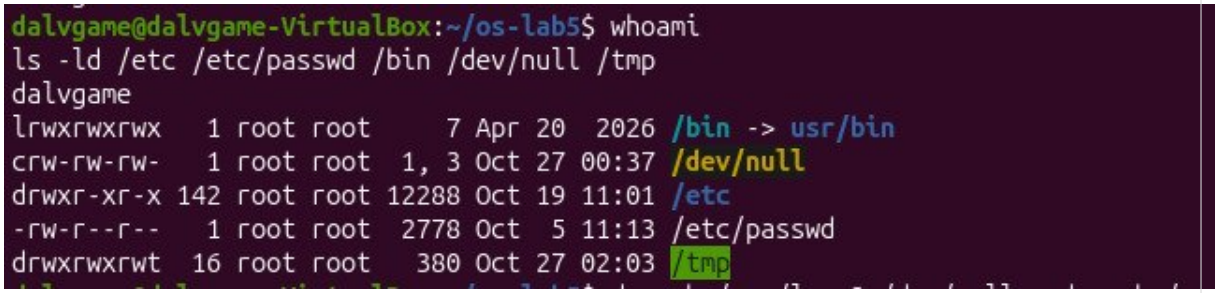


Figure 1.1 – Types from ls -ld and the /bin link

Comment and observation

The first character gives each type: /etc and /tmp are directories (d); /etc/passwd is a regular file (-); /bin is a symbolic link (l) to usr/bin; /dev/null is a character device (c). The later lsblk output shows /dev/sda as a disk and /dev/sda2 as its partition, which are block devices, although their ls -l type letters were not captured.

```
du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
```

The evidence for this exercise is shown in Figure 1.2.

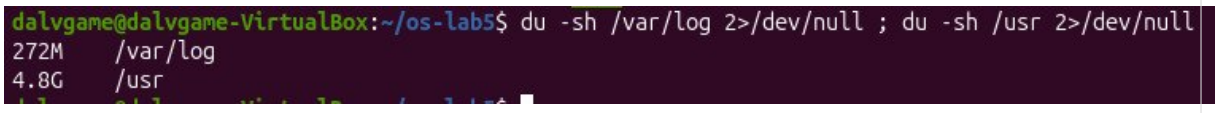


Figure 1.2 – Disk usage of two system trees

Comment and observation

/usr occupied about 4.8 GB, whereas /var/log occupied 192 MB: roughly 4.6 GB more, or about 26 times as much. The file command also identified /etc/passwd as ASCII text and /bin as a symbolic link.

2 INODES AND LINKS

This chapter follows Part 2 of the handout. The commands separate file names from inodes, then test hard links, symbolic links, directory link counts and allocated space.

2.1 Hard links and symbolic links

```
stat notes.txt ln notes.txt hard.txt ls -li notes.txt hard.txt rm notes.txt ; cat hard.txt
```

The evidence for this exercise is shown in Figure 2.1.

```
dalvgame@dalvgame-VirtualBox:~/os-lab5$ echo hello > notes.txt
stat notes.txt
ln notes.txt hard.txt
ls -li notes.txt hard.txt
rm notes.txt ; cat hard.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 1333040     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/dalvgame)   Gid: ( 1000/dalvgame)
Access: 2026-10-23 11:57:37.790389185 +0300
Modify: 2026-10-27 02:09:31.210660710 +0200
Change: 2026-10-27 02:09:31.210660710 +0200
 Birth: 2026-10-23 11:57:37.776388885 +0300
1333040 -rw-rw-r-- 2 dalvgame dalvgame 6 Oct 27 02:09 hard.txt
1333040 -rw-rw-r-- 2 dalvgame dalvgame 6 Oct 27 02:09 notes.txt
hello
```

Figure 2.1 – One inode shared by two hard-link names

Comment and observation

In stat, Inode is the inode number (1332217) and Links is the hard-link count. ln added a second directory entry for the same inode, increasing Links from 1 to 2. rm notes.txt removed only that name, so hard.txt still read the same six-byte contents and the count returned to 1.

```
ln -s hard.txt soft.txt ls -li hard.txt soft.txt cat soft.txt rm hard.txt ; cat soft.txt ; ls -l soft.txt
```

The evidence for this exercise is shown in Figure 2.2.

```
dalvgame@dalvgame-VirtualBox:~/os-lab5$ ln -s hard.txt soft.txt
ls -li hard.txt soft.txt
cat soft.txt
rm hard.txt ; cat soft.txt ; ls -l soft.txt
ln: Already exists
1333040 -rw-rw-r-- 1 dalvgame dalvgame 6 Oct 27 02:09 hard.txt
1332218 lrwxrwxrwx 1 dalvgame dalvgame 8 Oct 23 11:54 soft.txt -> hard.txt
hello
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 dalvgame dalvgame 8 Oct 23 11:54 soft.txt -> hard.txt
```

Figure 2.2 – The symbolic link became dangling

Comment and observation

soft.txt had its own inode (1332218) and stored the eight-byte target string hard.txt. Once that target name was removed, the link remained but following it failed. Its reported size is eight bytes because hard.txt contains eight characters.

2.2 Directory links and allocated space

```
mkdir d ; ls -ld d mkdir d/x d/y ; ls -ld d echo x > one.txt ; ls -l one.txt ; du -h one.txt truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
```

The evidence for this exercise is shown in Figure 2.3.

```
dalvgame@dalvgame-VirtualBox:~/os-lab5$ mkdir d ; ls -ld d
mkdir d/x d/y ; ls -ld d
echo x > one.txt ; ls -l one.txt ; du -h one.txt
truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
rm big.img
mkdir: d: File exists
drwxrwxr-x 4 dalvgame dalvgame 4096 Oct 23 11:55 d
mkdir: d/x: File exists
mkdir: d/y: File exists
drwxrwxr-x 4 dalvgame dalvgame 4096 Oct 23 11:55 d
-rw-rw-r-- 1 dalvgame dalvgame 2 Oct 27 02:11 one.txt
4.0K    one.txt
-rw-rw-r-- 1 dalvgame dalvgame 1.0G Oct 27 02:11 big.img
0       big.img
```

Figure 2.3 – Directory link counts and sparse allocation

Comment and observation

A new d has links for its parent directory entry and its own '.' entry. Each subdirectory's '..' entry points back to d, raising its count to 4. one.txt contains two bytes but occupies one 4 KiB allocation block. big.img has a logical length of 1 GiB but truncate left it sparse, so no data blocks were allocated and du reported zero.

3 PERMISSIONS

This chapter follows Part 3 of the handout. I changed file and directory modes as a normal user and checked how umask and special permission bits affect access.

3.1 File modes and umask

```
chmod 600 s.txt ; ls -l s.txt chmod 000 s.txt ; cat s.txt chmod u+r s.txt ; cat s.txt umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
```

The evidence for this exercise is shown in Figure 3.1.

```
dalvgame@dalvgame-VirtualBox:~/os-lab5$ echo secret > s.txt
chmod 600 s.txt ; ls -l s.txt
chmod 000 s.txt ; cat s.txt
chmod u+r s.txt ; cat s.txt
umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
bash: s.txt: Permission denied
-rw------- 1 dalvgame dalvgame 7 Oct 23 11:56 s.txt
cat: s.txt: Permission denied
secret
0002
mkdir: newdir: File exists
-rw-rw-r-- 1 dalvgame dalvgame    0 Oct 27 02:11 new.txt
drwxrwxr-x 2 dalvgame dalvgame 4096 Oct 23 11:56 newdir
```

Figure 3.1 – Read permission and default modes

Comment and observation

The mode strings are 600 = rw-------, 644 = rw-r--r--, 755 = rwxr-xr-x, and 700 = rwx------. Typical uses are private data, ordinary readable documents, public executable programs/directories, and private directories. Ownership identifies which permission class applies; it does not bypass chmod 000, so cat was denied. Umask 0002 removes write permission for others from creation defaults, yielding 664 for new.txt and 775 for newdir.

3.2 Directory search permission and special bits

chmod u=r,go= box ; ls box ; cat box/f chmod u=x,go= box ; ls box ; cat box/f ls -l /usr/bin/passwd ; ls -ld /tmp cat /etc/shadow

The evidence for this exercise is shown in Figure 3.2.

```
dalvgame@dalvgame-VirtualBox:~/os-lab5$ mkdir box ; echo hi > box/f
chmod u=r,go= box ; ls box ; cat box/f
chmod u=x,go= box ; ls box ; cat box/f
chmod 755 box
ls -l /usr/bin/passwd ; ls -ld /tmp
cat /etc/shadow
mkdir: box: File exists
f
cat: box/f: Permission denied
ls: cannot open directory 'box': Permission denied
hi
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 16 root root 380 Oct 27 02:03 /tmp
cat: /etc/shadow: Permission denied
```

Figure 3.2 – Directory r and x are different

Comment and observation

Directory r permits listing names, while x permits traversing to a known name. With only r, ls listed f but cat could not traverse box; with only x, cat box/f succeeded but ls could not list the directory. The s on passwd is setuid: it runs with the file owner's effective identity so it can update protected password data. The t on /tmp is the sticky bit: users may create files there, but normally cannot remove other users' files.

4 DISKS AND KERNEL FILES

This chapter follows Part 4 of the handout. I measured the Ubuntu guest's disk, root mount, CPU and memory, and read process information through /proc.

4.1 Disk and root file system

```
lsblk df -h mount | grep " / "
```

The evidence for this exercise is shown in Figure 4.1.

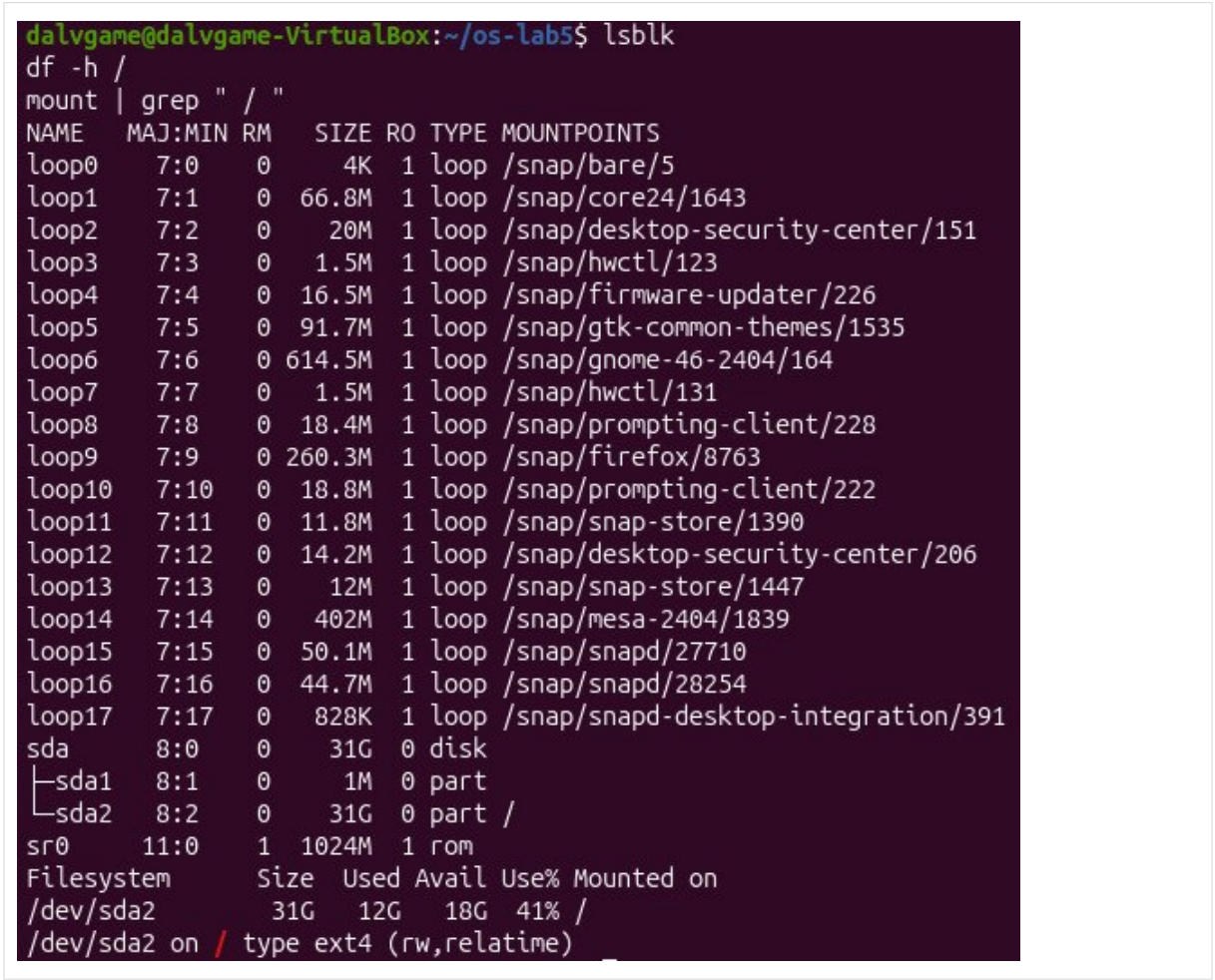


Figure 4.1 – The Ubuntu guest's disk and root mount

Comment and observation

The guest disk is /dev/sda (31 GB), with a 1 MB /dev/sda1 partition and a 31 GB /dev/sda2 partition. /dev/sda2 is mounted at / as ext4. At observation time it was 40% full (12 GB used and 18 GB available). The shared OS-labs folder shown elsewhere in df is a different mount, not the root file system.

4.2 Kernel-provided process information

```
grep -c processor /proc/cpuinfo head -3 /proc/meminfo head -5 /proc/self/status ls -l /proc/self/fd
```

The evidence for this exercise is shown in Figure 4.2.

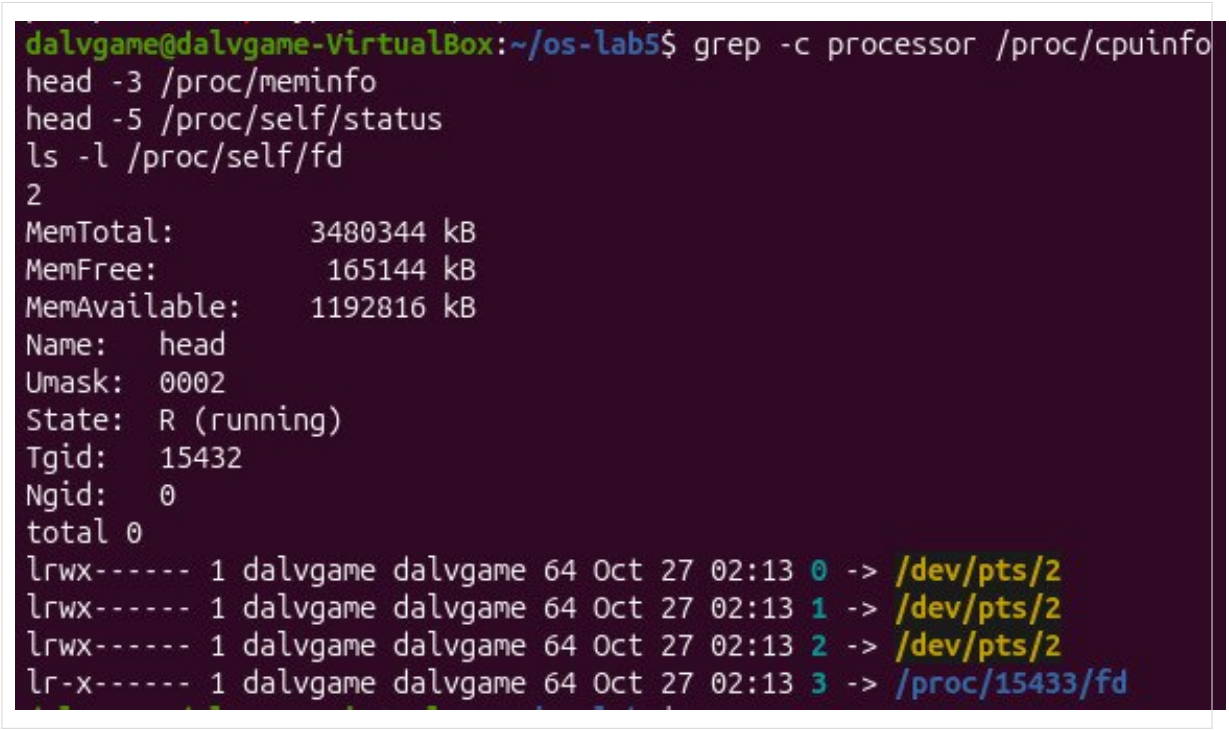


Figure 4.2 – CPU, memory, process status and descriptors

Comment and observation

/proc/cpuinfo reported 2 virtual processors and /proc/meminfo reported 3,480,328 kB of total memory (about 3.32 GiB) for the guest. /proc/self/status named head because head itself opened /proc/self/status; self resolves to the process reading it.

5 OPTIONAL EXPERIMENTS

I also ran the optional exercises for file signatures, deleted open files, setuid programs and a small ext4 image.

5.1 File signatures and deleted open files

```
gzip -c /etc/passwd > notes.txt ; file notes.txt xxd notes.txt | head -1 exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd exec 3<&-
```

The evidence for this exercise is shown in Figure 5.1.

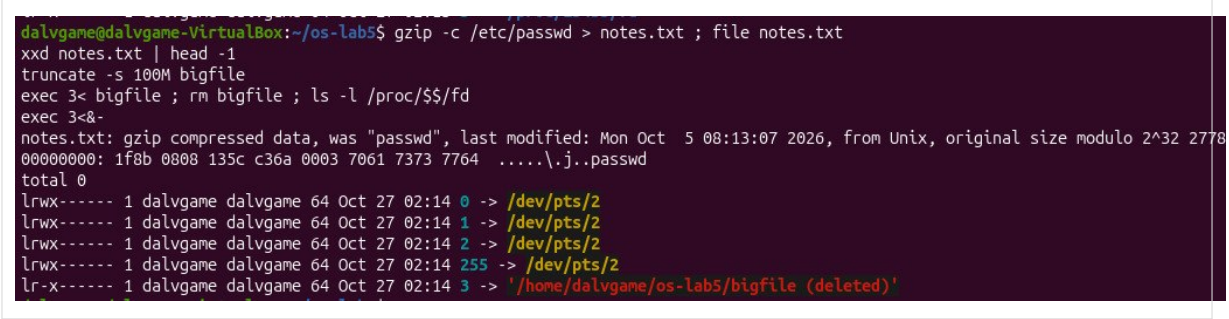


Figure 5.1 – Magic bytes and an open deleted file

Comment and observation

The leading bytes 1f 8b identify gzip despite the .txt suffix. After unlinking bigfile, descriptor 3 still pointed to an open inode marked '(deleted)'; the blocks become reclaimable after the last open reference closes.

Because bigfile was created with truncate and was sparse, df -h did not show a measurable change before or after closing it.

5.2 Setuid search and a temporary ext4 image

```
truncate -s 64M disk.img ; mkfs.ext4 disk.img sudo mount -o loop disk.img /mnt df -i /mnt sudo umount /mnt
```

The evidence for this exercise is shown in Figure 5.2.

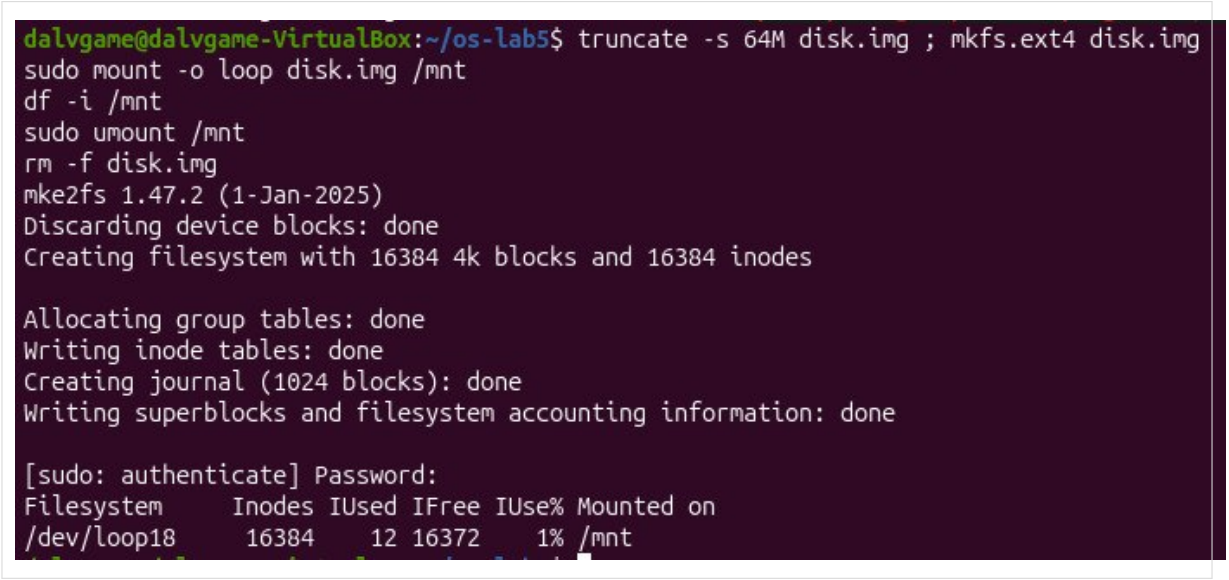


Figure 5.2 – The image contained 16,384 inodes

Comment and observation

The setuid search found passwd and several other programs, including sudo.ws, pkexec, ntfs-3g and ssh-keysign. The 64 MB ext4 image was formatted with 16,384 inodes; df -i showed 12 in use and 16,372 free after mounting. The recorded copy to the VirtualBox shared folder could not preserve soft.txt as a symbolic link, so an archive was created and a second copy omitted that link.

CONCLUSION

These experiments showed that a pathname is distinct from the inode it names. Hard links preserved access after one name was removed, while a symbolic link failed when its target disappeared. The inode numbers and link counts made this distinction visible in the terminal output.

The permission tests showed how file mode bits, directory traversal and the umask determine access. The sparse-file experiment showed that apparent size and allocated disk space can differ sharply. The disk and /proc commands exposed measurements of the Ubuntu guest and its running processes.

The optional ext4 image demonstrated a separate file system with its own blocks and inodes. These observations apply to the recorded Ubuntu virtual machine and the commands actually run in this lab.

REFERENCES

OS 2026 Lab 5 handout, The File System with Your Own Hands Linux, provided for this course.